```
const MODEL = {
  cpus: {
    "Ryzen 5 3400G": {
      vendor: "AMD",
      cores: 4,
      cacheClass: "standard",
      cacheBonus: -0.03,
      ryzenMemorySensitivity: 1.08,
      cpuFactor: 0.48,
      lowFactor: 0.38,
      confidence: "medium",
      notes: "APU vieja de 4/8; con GPUs potentes queda muy limitada en promedio y, sobre
todo, en 1% lows.",
    },
    "Ryzen 5 3500": {
      vendor: "AMD",
      cores: 6,
      cacheClass: "standard",
      cacheBonus: -0.01,
      ryzenMemorySensitivity: 1.04,
      cpuFactor: 0.72,
      lowFactor: 0.67,
      confidence: "medium",
      notes: "Cumple en juegos livianos, pero sus minimos quedan atras con GPUs
modernas.",
    },
    "Ryzen 5 4600G": {
      vendor: "AMD",
      cores: 6,
      cacheClass: "standard",
      cacheBonus: -0.01,
      ryzenMemorySensitivity: 1.02,
      cpuFactor: 0.79,
      lowFactor: 0.74,
      confidence: "medium",
      notes: "APU correcta para salir del paso, pero lejos de un 5600 en gaming serio.",
    },
    "Ryzen 5 3600": {
      vendor: "AMD",
      cores: 6,
      cacheClass: "standard",
      cacheBonus: 0,
      ryzenMemorySensitivity: 1,
      cpuFactor: 0.82,
      lowFactor: 0.78,
      confidence: "high",
      notes: "Todavia rinde bien, pero sus minimos sufren mas con GPUs modernas.",
    },
```

"Ryzen 5 3600X": {
vendor: "AMD",
cores: 6,
cacheClass: "standard",
cacheBonus: 0.01,
ryzenMemorySensitivity: 1,
cpuFactor: 0.84,
lowFactor: 0.8,
confidence: "medium",
notes: "Ligeramente mejor que el 3600, aunque sigue siendo una base ya veterana.",
},
"Ryzen 5 5500": {
vendor: "AMD",
cores: 6,
cacheClass: "standard",
cacheBonus: 0.01,
ryzenMemorySensitivity: 1,
cpuFactor: 0.87,
lowFactor: 0.84,
confidence: "medium",
notes: "Opcion de entrada AM4 correcta, pero por debajo de un 5600 completo en gaming.",
},
"Ryzen 5 5600": {
vendor: "AMD",
cores: 6,
cacheClass: "standard",
cacheBonus: 0.02,
ryzenMemorySensitivity: 1,
cpuFactor: 0.92,
lowFactor: 0.9,
confidence: "high",
notes: "Muy equilibrado para gaming y bastante agradecido con RAM 3200/3600.",
},
"Ryzen 5 5600X": {
vendor: "AMD",
cores: 6,
cacheClass: "standard",
cacheBonus: 0.03,
ryzenMemorySensitivity: 0.98,
cpuFactor: 0.95,
lowFactor: 0.92,
confidence: "high",
notes: "Una de las referencias clasicas de AM4 para jugar sin gastar demasiado.",
},
"Ryzen 5 5600X3D": {
vendor: "AMD",
cores: 6,

cacheClass: "x3d",
cacheBonus: 0.1,
ryzenMemorySensitivity: 0.72,
cpuFactor: 1,
lowFactor: 1,
confidence: "medium",
notes: "La cache 3D lo vuelve muy interesante para gaming, sobre todo en juegos CPU-bound.",
},
"Ryzen 5 5500X3D": {
vendor: "AMD",
cores: 6,
cacheClass: "x3d",
cacheBonus: 0.09,
ryzenMemorySensitivity: 0.74,
cpuFactor: 0.97,
lowFactor: 0.96,
confidence: "medium",
notes: "Seis nucleos con 3D V-Cache: muy buena mejora sobre el 5500 comun, sobre todo en juegos CPU-bound donde la cache extra marca la diferencia.",
},
"Ryzen 5 5600G": {
vendor: "AMD",
cores: 6,
cacheClass: "standard",
cacheBonus: -0.01,
ryzenMemorySensitivity: 1,
cpuFactor: 0.88,
lowFactor: 0.84,
confidence: "medium",
notes: "Recorta algo de rendimiento frente al 5600 por su configuracion de cache.",
},
"Ryzen 7 5700X": {
vendor: "AMD",
cores: 8,
cacheClass: "standard",
cacheBonus: 0.03,
ryzenMemorySensitivity: 0.95,
cpuFactor: 0.98,
lowFactor: 0.95,
confidence: "medium",
notes: "Los 8 nucleos ayudan en 1% lows, aunque no tiene el empuje de cache 3D.",
},
"Ryzen 7 3700X": {
vendor: "AMD",
cores: 8,
cacheClass: "standard",
cacheBonus: 0.01,

```
  ryzenMemorySensitivity: 1,
  cpuFactor: 0.86,
  lowFactor: 0.82,
  confidence: "medium",
  notes: "Todavia defendible, pero ya lejos de CPUs modernos en juegos CPU-bound.",
},
"Ryzen 7 3800X": {
  vendor: "AMD",
  cores: 8,
  cacheClass: "standard",
  cacheBonus: 0.01,
  ryzenMemorySensitivity: 0.98,
  cpuFactor: 0.88,
  lowFactor: 0.84,
  confidence: "medium",
  notes: "Algo mejor afinado que el 3700X, pero sigue en una liga claramente inferior.",
},
"Ryzen 7 5700X3D": {
  vendor: "AMD",
  cores: 8,
  cacheClass: "x3d",
  cacheBonus: 0.1,
  ryzenMemorySensitivity: 0.68,
  cpuFactor: 1.03,
  lowFactor: 1.03,
  confidence: "medium",
  notes: "Muy fuerte para AM4; la cache extra mejora bastante los juegos CPU-bound.",
},
"Ryzen 7 5800X3D": {
  vendor: "AMD",
  cores: 8,
  cacheClass: "x3d",
  cacheBonus: 0.12,
  ryzenMemorySensitivity: 0.65,
  cpuFactor: 1.08,
  lowFactor: 1.08,
  confidence: "high",
  notes: "La cache 3D mejora mucho juegos CPU-bound y estabiliza los minimos.",
},
"Ryzen 9 5900X": {
  vendor: "AMD",
  cores: 12,
  cacheClass: "standard",
  cacheBonus: 0.04,
  ryzenMemorySensitivity: 0.9,
  cpuFactor: 1,
  lowFactor: 0.98,
  confidence: "medium",
```

```
    notes: "Excelente para multitarea, aunque en gaming puro no supera a los X3D de AM4.",
  },
  "Ryzen 9 5950X": {
    vendor: "AMD",
    cores: 16,
    cacheClass: "standard",
    cacheBonus: 0.04,
    ryzenMemorySensitivity: 0.88,
    cpuFactor: 1.01,
    lowFactor: 0.99,
    confidence: "medium",
    notes: "Brilla mas en productividad que en juegos frente a opciones X3D.",
  },
  "Ryzen 5 7500F": {
    vendor: "AMD",
    cores: 6,
    cacheClass: "standard",
    cacheBonus: 0.04,
    ryzenMemorySensitivity: 0.7,
    cpuFactor: 1.02,
    lowFactor: 0.99,
    confidence: "medium",
    notes: "Muy buen valor en AM5, bastante cercano al 7600 en gaming.",
  },
  "Ryzen 5 7600": {
    vendor: "AMD",
    cores: 6,
    cacheClass: "standard",
    cacheBonus: 0.04,
    ryzenMemorySensitivity: 0.7,
    cpuFactor: 1.04,
    lowFactor: 1.01,
    confidence: "high",
    notes: "Zen 4 levanta bastante el promedio aun con 6 nucleos.",
  },
  "Ryzen 5 7600X": {
    vendor: "AMD",
    cores: 6,
    cacheClass: "standard",
    cacheBonus: 0.05,
    ryzenMemorySensitivity: 0.68,
    cpuFactor: 1.06,
    lowFactor: 1.03,
    confidence: "high",
    notes: "Sube un poco mas el techo de FPS que el 7600 comun.",
  },
  "Ryzen 7 7700": {
```

vendor: "AMD",
cores: 8,
cacheClass: "standard",
cacheBonus: 0.06,
ryzenMemorySensitivity: 0.68,
cpuFactor: 1.08,
lowFactor: 1.05,
confidence: "high",
notes: "Muy solido en gaming general y con buenos minimos.",
},
"Ryzen 7 7700X": {
vendor: "AMD",
cores: 8,
cacheClass: "standard",
cacheBonus: 0.07,
ryzenMemorySensitivity: 0.66,
cpuFactor: 1.1,
lowFactor: 1.07,
confidence: "high",
notes: "Muy rapido en gaming clasico, aunque por detras de los X3D en juegos CPU-bound.",
},
"Ryzen 7 7800X3D": {
vendor: "AMD",
cores: 8,
cacheClass: "x3d",
cacheBonus: 0.16,
ryzenMemorySensitivity: 0.55,
cpuFactor: 1.16,
lowFactor: 1.16,
confidence: "high",
notes: "Referencia muy fuerte para gaming puro, sobre todo en juegos CPU-bound.",
},
"Ryzen 9 7900X": {
vendor: "AMD",
cores: 12,
cacheClass: "standard",
cacheBonus: 0.08,
ryzenMemorySensitivity: 0.6,
cpuFactor: 1.1,
lowFactor: 1.07,
confidence: "medium",
notes: "Muy buen CPU generalista, aunque no tan fino para gaming como un 7800X3D.",
},
"Ryzen 9 7900X3D": {
vendor: "AMD",
cores: 12,

cacheClass: "x3d",
cacheBonus: 0.14,
ryzenMemorySensitivity: 0.54,
cpuFactor: 1.13,
lowFactor: 1.12,
confidence: "medium",
notes: "Muy fuerte, aunque normalmente el 7800X3D sigue siendo referencia mas limpia en gaming.",
},
"Ryzen 9 7950X3D": {
vendor: "AMD",
cores: 16,
cacheClass: "x3d",
cacheBonus: 0.17,
ryzenMemorySensitivity: 0.5,
cpuFactor: 1.17,
lowFactor: 1.17,
confidence: "medium",
notes: "Tope AMD muy capaz, con gran techo tanto en gaming como en tareas mixtas.",
},
"Ryzen 9 9900X3D": {
vendor: "AMD",
cores: 12,
cacheClass: "x3d",
cacheBonus: 0.19,
ryzenMemorySensitivity: 0.48,
cpuFactor: 1.19,
lowFactor: 1.18,
confidence: "low",
notes: "Estimado con prudencia como una evolucion moderna de la familia X3D.",
},
"Ryzen 7 9700X": {
vendor: "AMD",
cores: 8,
cacheClass: "standard",
cacheBonus: 0.09,
ryzenMemorySensitivity: 0.54,
cpuFactor: 1.13,
lowFactor: 1.1,
confidence: "low",
notes: "CPU moderno con buen IPC, estimado de forma prudente dentro del modelo.",
},
"Ryzen 7 9800X3D": {
vendor: "AMD",
cores: 8,
cacheClass: "x3d",
cacheBonus: 0.19,
ryzenMemorySensitivity: 0.48,

```
    cpuFactor: 1.2,
    lowFactor: 1.2,
    confidence: "low",
    notes: "Estimado conservadoramente como una mejora corta sobre el 7800X3D.",
  },
  "Core i3-12100F": {
    vendor: "Intel",
    cores: 4,
    cacheClass: "standard",
    cacheBonus: -0.01,
    ryzenMemorySensitivity: 0.4,
    cpuFactor: 0.76,
    lowFactor: 0.7,
    confidence: "medium",
    notes: "Sorprende en esports, pero se queda corto antes en titulos pesados.",
  },
  "Core i5-11400F": {
    vendor: "Intel",
    cores: 6,
    cacheClass: "standard",
    cacheBonus: -0.01,
    ryzenMemorySensitivity: 0.44,
    cpuFactor: 0.8,
    lowFactor: 0.76,
    confidence: "medium",
    notes: "Correcto para gama media vieja, pero claramente superado por generaciones
mas nuevas.",
  },
  "Core i7-11700K": {
    vendor: "Intel",
    cores: 8,
    cacheClass: "standard",
    cacheBonus: 0.01,
    ryzenMemorySensitivity: 0.4,
    cpuFactor: 0.87,
    lowFactor: 0.83,
    confidence: "medium",
    notes: "Aun usable, aunque queda por detras de varios Ryzen 5000 en gaming.",
  },
  "Core i9-11900K": {
    vendor: "Intel",
    cores: 8,
    cacheClass: "standard",
    cacheBonus: 0.02,
    ryzenMemorySensitivity: 0.38,
    cpuFactor: 0.89,
    lowFactor: 0.85,
    confidence: "medium",
```

```
  notes: "Fue tope de gama, pero hoy ya no destaca frente a opciones mas modernas.",
},
"Core i5-12400F": {
  vendor: "Intel",
  cores: 6,
  cacheClass: "standard",
  cacheBonus: 0.01,
  ryzenMemorySensitivity: 0.45,
  cpuFactor: 0.9,
  lowFactor: 0.87,
  confidence: "high",
  notes: "Muy capaz en rango medio, con buen comportamiento general.",
},
"Core i5-14400F": {
  vendor: "Intel",
  cores: 10,
  cacheClass: "standard",
  cacheBonus: 0.03,
  ryzenMemorySensitivity: 0.36,
  cpuFactor: 0.97,
  lowFactor: 0.94,
  confidence: "medium",
  notes: "Opcion moderna y equilibrada para gaming sin irse a un K.",
},
"Core i5-12600K": {
  vendor: "Intel",
  cores: 10,
  cacheClass: "standard",
  cacheBonus: 0.02,
  ryzenMemorySensitivity: 0.4,
  cpuFactor: 0.96,
  lowFactor: 0.93,
  confidence: "high",
  notes: "Sigue siendo una base muy valida para gaming con GPUs medias o altas.",
},
"Core i7-12700K": {
  vendor: "Intel",
  cores: 12,
  cacheClass: "standard",
  cacheBonus: 0.03,
  ryzenMemorySensitivity: 0.34,
  cpuFactor: 1.01,
  lowFactor: 0.99,
  confidence: "high",
  notes: "Mantiene bien el tipo cuando hay mucha carga de fondo o escenas pesadas.",
},
"Core i9-12900K": {
  vendor: "Intel",
```

```
  cores: 16,
  cacheClass: "standard",
  cacheBonus: 0.04,
  ryzenMemorySensitivity: 0.34,
  cpuFactor: 1.03,
  lowFactor: 1,
  confidence: "medium",
  notes: "Aun fuerte, aunque los X3D modernos lo superan en varios juegos.",
},
"Core i5-13400F": {
  vendor: "Intel",
  cores: 10,
  cacheClass: "standard",
  cacheBonus: 0.02,
  ryzenMemorySensitivity: 0.38,
  cpuFactor: 0.94,
  lowFactor: 0.9,
  confidence: "medium",
  notes: "Buen punto medio para builds gamer de presupuesto controlado.",
},
"Core i5-13600K": {
  vendor: "Intel",
  cores: 14,
  cacheClass: "standard",
  cacheBonus: 0.04,
  ryzenMemorySensitivity: 0.35,
  cpuFactor: 1.08,
  lowFactor: 1.06,
  confidence: "high",
  notes: "Muy rapido en esports y fuerte en cargas mixtas.",
},
"Core i7-13700K": {
  vendor: "Intel",
  cores: 16,
  cacheClass: "standard",
  cacheBonus: 0.05,
  ryzenMemorySensitivity: 0.32,
  cpuFactor: 1.1,
  lowFactor: 1.08,
  confidence: "high",
  notes: "Muy consistente con GPUs top y gran techo de FPS.",
},
"Core i9-13900K": {
  vendor: "Intel",
  cores: 24,
  cacheClass: "standard",
  cacheBonus: 0.06,
  ryzenMemorySensitivity: 0.3,
```

```
    cpuFactor: 1.12,
    lowFactor: 1.1,
    confidence: "medium",
    notes: "Muy rapido, pero con ventaja menor en gaming frente a CPUs mejor afinados.",
  },
  "Core i5-14600K": {
    vendor: "Intel",
    cores: 14,
    cacheClass: "standard",
    cacheBonus: 0.05,
    ryzenMemorySensitivity: 0.32,
    cpuFactor: 1.11,
    lowFactor: 1.08,
    confidence: "medium",
    notes: "Escalon muy fuerte para gaming de alta tasa de refresco.",
  },
  "Core i7-14700K": {
    vendor: "Intel",
    cores: 20,
    cacheClass: "standard",
    cacheBonus: 0.06,
    ryzenMemorySensitivity: 0.3,
    cpuFactor: 1.14,
    lowFactor: 1.12,
    confidence: "medium",
    notes: "Muy fuerte en promedio y minimos, especialmente con GPUs tope de gama.",
  },
  "Core i9-14900K": {
    vendor: "Intel",
    cores: 24,
    cacheClass: "standard",
    cacheBonus: 0.07,
    ryzenMemorySensitivity: 0.28,
    cpuFactor: 1.15,
    lowFactor: 1.13,
    confidence: "medium",
    notes: "Tope Intel muy rapido, aunque con retornos decrecientes en juegos puros.",
  },
},
gpus: {
  "GTX 1660 Super": {
    gpuFactor: 0.42,
    lowFactor: 0.41,
    saturationBias: 8,
    confidence: "medium",
    notes: "Sigue viva para 1080p bajo/medio, pero ya muy lejos de la gama actual.",
  },
  "RTX 3050": {
```

```
  gpuFactor: 0.5,
  lowFactor: 0.48,
  saturationBias: 7,
  confidence: "medium",
  notes: "Entrada moderna de NVIDIA, claramente orientada a 1080p.",
},
"RTX 2060": {
  gpuFactor: 0.56,
  lowFactor: 0.55,
  saturationBias: 6,
  confidence: "medium",
  notes: "Gama media vieja, mejor pensada para 1080p ajustado.",
},
"RTX 2070 Super": {
  gpuFactor: 0.69,
  lowFactor: 0.67,
  saturationBias: 5,
  confidence: "medium",
  notes: "Aun defendible para 1080p alto y 1440p recortado.",
},
"RTX 2080": {
  gpuFactor: 0.74,
  lowFactor: 0.72,
  saturationBias: 4,
  confidence: "medium",
  notes: "Vieja gama alta que hoy se ubica en una zona media aceptable.",
},
"RTX 2080 Super": {
  gpuFactor: 0.78,
  lowFactor: 0.76,
  saturationBias: 3,
  confidence: "medium",
  notes: "Algo mejor afinada para 1440p que la 2080 base.",
},
"RTX 2080 Ti": {
  gpuFactor: 0.84,
  lowFactor: 0.82,
  saturationBias: 2,
  confidence: "medium",
  notes: "Todavia bastante competitiva en raster, pese a su edad.",
},
"RTX 3060": {
  gpuFactor: 0.72,
  lowFactor: 0.71,
  saturationBias: 4,
  confidence: "high",
  notes: "Muy comun para 1080p y 1440p moderado.",
},
```

```
"RTX 3060 Ti": {
  gpuFactor: 0.8,
  lowFactor: 0.78,
  saturationBias: 3,
  confidence: "high",
  notes: "Sube bien sobre la 3060 y sigue siendo muy capaz para 1440p moderado.",
},
"RTX 4060": {
  gpuFactor: 0.76,
  lowFactor: 0.74,
  saturationBias: 4,
  confidence: "medium",
  notes: "Competente en 1080p, aunque no destaca por fuerza bruta.",
},
"RTX 3070": {
  gpuFactor: 0.88,
  lowFactor: 0.86,
  saturationBias: 2,
  confidence: "high",
  notes: "Sigue siendo fuerte en raster para 1440p.",
},
"RTX 3070 Ti": {
  gpuFactor: 0.92,
  lowFactor: 0.9,
  saturationBias: 1,
  confidence: "medium",
  notes: "Algo mejor que la 3070, pero sin un salto enorme.",
},
"RTX 3080": {
  gpuFactor: 1,
  lowFactor: 1,
  saturationBias: 0,
  confidence: "high",
  notes: "GPU de referencia para las bases del modelo.",
},
"RTX 3080 Ti": {
  gpuFactor: 1.1,
  lowFactor: 1.08,
  saturationBias: -1,
  confidence: "medium",
  notes: "Muy competente aun hoy, especialmente en raster alto.",
},
"RTX 3090": {
  gpuFactor: 1.12,
  lowFactor: 1.1,
  saturationBias: -1,
  confidence: "medium",
  notes: "Muy cercana a 3080 Ti en gaming, con mas VRAM para escenarios concretos.",
```

},
"RTX 3090 Ti": {
  gpuFactor: 1.18,
  lowFactor: 1.16,
  saturationBias: -2,
  confidence: "medium",
  notes: "Version mas extrema de Ampere, todavia muy rapida en 1440p y 4K.",
},
"RTX 4060 Ti": {
  gpuFactor: 0.84,
  lowFactor: 0.82,
  saturationBias: 2,
  confidence: "medium",
  notes: "Gana algo frente a la 4060, pero sigue sin ser una bestia de raster.",
},
"RTX 4070": {
  gpuFactor: 1,
  lowFactor: 0.98,
  saturationBias: 0,
  confidence: "high",
  notes: "Muy buena para 1440p y bastante eficiente.",
},
"RTX 4070 Super": {
  gpuFactor: 1.08,
  lowFactor: 1.07,
  saturationBias: -1,
  confidence: "high",
  notes: "Excelente salto en eficiencia y buen techo para 1440p.",
},
"RTX 4070 Ti Super": {
  gpuFactor: 1.17,
  lowFactor: 1.15,
  saturationBias: -2,
  confidence: "medium",
  notes: "Muy fuerte en 1440p y bastante mas holgada en 4K que una 4070 Super.",
},
"RTX 4080 Super": {
  gpuFactor: 1.24,
  lowFactor: 1.22,
  saturationBias: -4,
  confidence: "medium",
  notes: "Muy rapida; expone antes los limites del CPU a 1080p.",
},
"RTX 4090": {
  gpuFactor: 1.38,
  lowFactor: 1.35,
  saturationBias: -7,
  confidence: "medium",

```
  notes: "Extremo superior; a 1080p muchas veces el cuello pasa a ser el CPU.",
},
"RX 6600": {
  gpuFactor: 0.66,
  lowFactor: 0.64,
  saturationBias: 5,
  confidence: "medium",
  notes: "Buena para 1080p, pero limitada para presets altos en juegos pesados.",
},
"RX 6650 XT": {
  gpuFactor: 0.72,
  lowFactor: 0.7,
  saturationBias: 4,
  confidence: "medium",
  notes: "Muy buena relacion precio/rendimiento para 1080p alto.",
},
"RX 6700 XT": {
  gpuFactor: 0.84,
  lowFactor: 0.82,
  saturationBias: 2,
  confidence: "high",
  notes: "Muy buena opcion para 1080p alto y 1440p razonable.",
},
"RX 6750 XT": {
  gpuFactor: 0.87,
  lowFactor: 0.85,
  saturationBias: 2,
  confidence: "medium",
  notes: "Pequeno extra sobre 6700 XT, util sobre todo en raster.",
},
"RX 6800": {
  gpuFactor: 0.96,
  lowFactor: 0.94,
  saturationBias: 1,
  confidence: "medium",
  notes: "Muy decente para 1440p y aun capaz a 4K con ajustes razonables.",
},
"RX 6800 XT": {
  gpuFactor: 1.02,
  lowFactor: 1,
  saturationBias: 0,
  confidence: "medium",
  notes: "Sigue siendo muy valida para 1440p y raster pesado.",
},
"RX 7700 XT": {
  gpuFactor: 0.96,
  lowFactor: 0.94,
  saturationBias: 1,
```

```
  confidence: "medium",
  notes: "Alternativa moderna para 1440p medio/alto.",
},
"RX 6950 XT": {
  gpuFactor: 1.1,
  lowFactor: 1.08,
  saturationBias: -1,
  confidence: "medium",
  notes: "Muy fuerte en raster, con algo mas de margen que la 6800 XT.",
},
"RX 7600": {
  gpuFactor: 0.71,
  lowFactor: 0.69,
  saturationBias: 4,
  confidence: "medium",
  notes: "Pensada para 1080p; en juegos pesados se queda antes que una 6700 XT.",
},
"RX 7600 XT": {
  gpuFactor: 0.74,
  lowFactor: 0.72,
  saturationBias: 4,
  confidence: "low",
  notes: "Muy cercana a la 7600 comun en gaming puro dentro de este modelo.",
},
"RX 7800 XT": {
  gpuFactor: 1.04,
  lowFactor: 1.02,
  saturationBias: 0,
  confidence: "medium",
  notes: "Muy capaz en raster a 1440p y con VRAM amplia.",
},
"RX 7900 GRE": {
  gpuFactor: 1.08,
  lowFactor: 1.06,
  saturationBias: -1,
  confidence: "medium",
  notes: "Se ubica entre 7800 XT y 7900 XT con buen valor en raster.",
},
"RX 7900 XT": {
  gpuFactor: 1.18,
  lowFactor: 1.16,
  saturationBias: -2,
  confidence: "medium",
  notes: "Muy fuerte en 1440p y 4K nativo dentro de raster.",
},
"RX 7900 XTX": {
  gpuFactor: 1.28,
  lowFactor: 1.25,
```

```
    saturationBias: -5,
    confidence: "medium",
    notes: "Muy potente en raster; en 1080p depende mucho del CPU.",
  },
},
ram: {
  capacity: {
    "8 GB": { avgModifier: -0.14, lowModifier: -0.18, confidencePenalty: 1 },
    "16 GB": { avgModifier: 0, lowModifier: 0, confidencePenalty: 0 },
    "32 GB": { avgModifier: 0.02, lowModifier: 0.03, confidencePenalty: 0 },
  },
  speed: {
    "2666 MHz": { baseModifier: -0.05 },
    "3000 MHz": { baseModifier: -0.02 },
    "3200 MHz": { baseModifier: 0 },
    "3600 MHz": { baseModifier: 0.03 },
  },
  channel: {
    "Single channel": { avgModifier: -0.09, lowModifier: -0.12, confidencePenalty: 1 },
    "Dual channel": { avgModifier: 0, lowModifier: 0, confidencePenalty: 0 },
  },
},
resolutions: {
  "1080p": { cpuScale: 1, gpuScale: 1.12, usageDelta: -8 },
  "1440p": { cpuScale: 0.95, gpuScale: 1, usageDelta: 0 },
  "4K": { cpuScale: 0.9, gpuScale: 0.66, usageDelta: 10 },
},
games: {
  CS2: {
    category: "CPU-bound",
    confidence: "high",
    x3dWeight: 1,
    ramWeight: 1,
    notes: "Shooter muy sensible a cache, latencia y frametimes.",
    presets: {
      competitive: {
        cpuBaseline: { avg: 330, low1: 210, gpuUsage: 74 },
        gpuBaseline: { avg: 290, low1: 210, gpuUsage: 97 },
        cpuClamp: { min: 120, max: 520 },
        gpuClamp: { min: 110, max: 420 },
      },
      ultra: {
        cpuBaseline: { avg: 285, low1: 190, gpuUsage: 80 },
        gpuBaseline: { avg: 240, low1: 175, gpuUsage: 98 },
        cpuClamp: { min: 100, max: 440 },
        gpuClamp: { min: 90, max: 340 },
      },
    },
```

},
Warzone: {
  category: "CPU-bound",
  confidence: "medium",
  x3dWeight: 0.95,
  ramWeight: 0.9,
  notes: "Warzone suele castigar CPU, RAM y 1% lows, sobre todo en 1080p.",
  presets: {
    competitive: {
      cpuBaseline: { avg: 178, low1: 128, gpuUsage: 81 },
      gpuBaseline: { avg: 168, low1: 122, gpuUsage: 98 },
      cpuClamp: { min: 70, max: 210 },
      gpuClamp: { min: 65, max: 195 },
    },
    ultra: {
      cpuBaseline: { avg: 154, low1: 108, gpuUsage: 86 },
      gpuBaseline: { avg: 129, low1: 94, gpuUsage: 99 },
      cpuClamp: { min: 60, max: 180 },
      gpuClamp: { min: 52, max: 155 },
    },
  },
},
Fortnite: {
  category: "CPU-bound",
  confidence: "medium",
  x3dWeight: 0.9,
  ramWeight: 0.85,
  notes: "Con ajustes competitivos suele escalar mucho con CPU, RAM rapida y cache.",
  presets: {
    competitive: {
      cpuBaseline: { avg: 232, low1: 162, gpuUsage: 78 },
      gpuBaseline: { avg: 224, low1: 165, gpuUsage: 97 },
      cpuClamp: { min: 90, max: 320 },
      gpuClamp: { min: 90, max: 290 },
    },
    ultra: {
      cpuBaseline: { avg: 174, low1: 122, gpuUsage: 84 },
      gpuBaseline: { avg: 146, low1: 106, gpuUsage: 99 },
      cpuClamp: { min: 70, max: 230 },
      gpuClamp: { min: 55, max: 180 },
    },
  },
},
"Cyberpunk 2077": {
  category: "GPU-bound",
  confidence: "high",
  x3dWeight: 0.35,
  ramWeight: 0.35,

```
    notes: "Juego muy pesado del lado GPU; la CPU importa menos salvo en 1080p.",
    presets: {
      competitive: {
        cpuBaseline: { avg: 141, low1: 105, gpuUsage: 89 },
        gpuBaseline: { avg: 118, low1: 90, gpuUsage: 98 },
        cpuClamp: { min: 65, max: 180 },
        gpuClamp: { min: 45, max: 150 },
      },
      ultra: {
        cpuBaseline: { avg: 117, low1: 88, gpuUsage: 92 },
        gpuBaseline: { avg: 86, low1: 67, gpuUsage: 99 },
        cpuClamp: { min: 52, max: 145 },
        gpuClamp: { min: 32, max: 110 },
      },
    },
  },
  "Alan Wake 2": {
    category: "GPU-bound",
    confidence: "medium",
    x3dWeight: 0.25,
    ramWeight: 0.25,
    notes: "Muy dependiente de GPU y memoria de video; el CPU suele pasar a segundo
plano.",
    presets: {
      competitive: {
        cpuBaseline: { avg: 108, low1: 81, gpuUsage: 91 },
        gpuBaseline: { avg: 82, low1: 63, gpuUsage: 99 },
        cpuClamp: { min: 48, max: 138 },
        gpuClamp: { min: 30, max: 110 },
      },
      ultra: {
        cpuBaseline: { avg: 96, low1: 72, gpuUsage: 93 },
        gpuBaseline: { avg: 66, low1: 52, gpuUsage: 99 },
        cpuClamp: { min: 40, max: 122 },
        gpuClamp: { min: 24, max: 88 },
      },
    },
  },
  "GTA V": {
    category: "Mixed",
    confidence: "medium",
    x3dWeight: 0.55,
    ramWeight: 0.45,
    notes: "Puede volverse CPU-bound a FPS muy altos, pero tambien escala con GPU.",
    presets: {
      competitive: {
        cpuBaseline: { avg: 196, low1: 142, gpuUsage: 84 },
        gpuBaseline: { avg: 188, low1: 144, gpuUsage: 98 },
```

```
        cpuClamp: { min: 90, max: 260 },
        gpuClamp: { min: 85, max: 245 },
      },
      ultra: {
        cpuBaseline: { avg: 168, low1: 124, gpuUsage: 88 },
        gpuBaseline: { avg: 142, low1: 110, gpuUsage: 99 },
        cpuClamp: { min: 72, max: 220 },
        gpuClamp: { min: 58, max: 185 },
      },
    },
  },
  "Red Dead Redemption 2": {
    category: "Mixed",
    confidence: "high",
    x3dWeight: 0.42,
    ramWeight: 0.4,
    notes: "Muy equilibrado: a 1080p influye el CPU, pero a 1440p y 4K manda mas la
GPU.",
    presets: {
      competitive: {
        cpuBaseline: { avg: 146, low1: 108, gpuUsage: 87 },
        gpuBaseline: { avg: 132, low1: 102, gpuUsage: 98 },
        cpuClamp: { min: 62, max: 182 },
        gpuClamp: { min: 50, max: 165 },
      },
      ultra: {
        cpuBaseline: { avg: 126, low1: 92, gpuUsage: 90 },
        gpuBaseline: { avg: 108, low1: 84, gpuUsage: 99 },
        cpuClamp: { min: 54, max: 158 },
        gpuClamp: { min: 40, max: 136 },
      },
    },
  },
 },
};

const state = {
  preset: "competitive",
};

const cpuSelect = document.querySelector("#cpu-select");
const gpuSelect = document.querySelector("#gpu-select");
const cpuSearch = document.querySelector("#cpu-search");
const gpuSearch = document.querySelector("#gpu-search");
const gameSelect = document.querySelector("#game-select");
const resolutionSelect = document.querySelector("#resolution-select");
const ramCapacitySelect = document.querySelector("#ram-capacity-select");
const ramSpeedSelect = document.querySelector("#ram-speed-select");
```

```
const ramChannelSelect = document.querySelector("#ram-channel-select");
const compareCpuA = document.querySelector("#compare-cpu-a");
const compareCpuB = document.querySelector("#compare-cpu-b");
const compareCpuASearch = document.querySelector("#compare-cpu-a-search");
const compareCpuBSearch = document.querySelector("#compare-cpu-b-search");
const diagnosisText = document.querySelector("#diagnosis-text");
const confidencePill = document.querySelector("#confidence-pill");
const avgFpsValue = document.querySelector("#avg-fps-value");
const rangeValue = document.querySelector("#range-value");
const lowValue = document.querySelector("#low-value");
const gpuUsageValue = document.querySelector("#gpu-usage-value");
const explanationText = document.querySelector("#explanation-text");
const referenceText = document.querySelector("#reference-text");
const resultCard = document.querySelector("#result-card");
const compareResults = document.querySelector("#compare-results");
const avgBar = document.querySelector("#avg-bar");
const lowBar = document.querySelector("#low-bar");
const gpuBar = document.querySelector("#gpu-bar");
const presetButtons = document.querySelectorAll(".preset-btn");
const bottleneckGauge = document.querySelector("#bottleneck-gauge");
const bottleneckPercent = document.querySelector("#bottleneck-percent");
const hardwareOptions = {
  cpus: Object.keys(MODEL.cpus).sort((a, b) => a.localeCompare(b)),
  gpus: Object.keys(MODEL.gpus).sort((a, b) => a.localeCompare(b)),
};

function clamp(value, min, max) {
  return Math.min(Math.max(value, min), max);
}

function fillSelect(selectElement, options, preferredValue) {
  selectElement.innerHTML = "";

  options.forEach((option) => {
    const item = document.createElement("option");
    item.value = option;
    item.textContent = option;
    selectElement.appendChild(item);
  });

  if (options.length === 0) {
    const fallback = document.createElement("option");
    fallback.value = "";
    fallback.textContent = "Sin resultados";
    selectElement.appendChild(fallback);
    selectElement.disabled = true;
    return;
  }
```

```
  selectElement.disabled = false;

  if (preferredValue && options.includes(preferredValue)) {
    selectElement.value = preferredValue;
    return;
  }

  if (!options.includes(selectElement.value)) {
    selectElement.value = options[0];
  }
}

function filterOptions(options, query) {
  const normalizedQuery = query.trim().toLowerCase();

  if (!normalizedQuery) {
    return options;
  }

  return options.filter((option) => option.toLowerCase().includes(normalizedQuery));
}

function refreshHardwareSelect(selectElement, sourceOptions, query, preferredValue) {
  const filtered = filterOptions(sourceOptions, query);
  fillSelect(selectElement, filtered, preferredValue);
  return filtered.length > 0;
}

function getSuggestionLabel(option, type) {
  if (type === "cpu") {
    const cpu = MODEL.cpus[option];
    return `${cpu.vendor} • ${cpu.cores} nucleos`;
  }

  const gpu = MODEL.gpus[option];
  return `${gpu.confidence === "high" ? "Referencia fuerte" : "Estimacion conservadora"}`;
}

function createSuggestionList(input) {
  const list = document.createElement("ul");
  list.className = "search-suggestions";
  list.hidden = true;
  input.parentElement.appendChild(list);
  return list;
}

function renderSuggestions(listElement, matches, type, onSelect) {
```

```
  listElement.innerHTML = "";

  if (matches.length === 0) {
    listElement.hidden = true;
    return;
  }

  matches.slice(0, 7).forEach((match, index) => {
    const listItem = document.createElement("li");
    const button = document.createElement("button");
    button.type = "button";
    button.className = "suggestion-item";
    if (index === 0) {
      button.classList.add("is-active");
    }

    button.innerHTML = `${match}<small>${getSuggestionLabel(match, type)}</small>`;
    button.addEventListener("mousedown", (event) => {
      event.preventDefault();
      onSelect(match);
    });
    listItem.appendChild(button);
    listElement.appendChild(listItem);
  });

  listElement.hidden = false;
}

// ——— CALCULATION ENGINE v2 ———————————————————————————————————————————————
//
// Modelo fisicamente coherente:
//
//  1. Techo GPU (gpuCeiling): cuantos FPS puede entregar la GPU a plena carga
//     en este juego/resolucion/preset. Es independiente del CPU.
//
//  2. Techo CPU (cpuCeiling): cuantos FPS puede "alimentar" el CPU al render
//     thread. Depende de cache, nucleos, RAM y sensibilidad del juego al CPU.
//
//  3. GPU Usage real: si cpuCeiling < gpuCeiling, la GPU espera frames del CPU
//     y su uso baja proporcionalmente. Si gpuCeiling <= cpuCeiling, la GPU
//     corre al maximo y su uso es ~97-99%.
//
//  4. FPS final: no es min() duro sino una combinacion ponderada por
//     gameCpuWeight para reflejar que el juego nunca es 100% CPU ni 100% GPU.
//
//  5. 1% lows: modelados desde la varianza de frame time del componente
//     limitante. CPU-bound => mayor varianza => lows mas bajos relativos.
```

```
//     GPU-bound => frame time mas estable => lows mas altos relativos.
//
//  6. Bottleneck %: diferencia relativa entre ambos techos. Positivo = CPU
//     limita, negativo = GPU limita. Cero = perfectamente balanceado.
//
//
──────────────
```

```
  // gpuUsage cuando esta GPU-limited: baseline + sesgo de saturacion del modelo
  // saturationBias negativo = GPU muy potente, no llega a saturarse facil
  const baseUsage = clamp(preset.gpuBaseline.gpuUsage + gpu.saturationBias, 85, 99);

  return {
    avg:      clamp(Math.round(avgRaw), preset.gpuClamp.min, preset.gpuClamp.max),
    low1:     clamp(Math.round(lowRaw), Math.round(preset.gpuClamp.min * 0.72),
Math.round(preset.gpuClamp.max * 0.84)),
    fullUsage: baseUsage,
  };
}

// Techo de CPU: cuantos FPS puede alimentar el CPU al render thread.
// Incluye efecto de X3D cache, nucleos extra, y RAM.
function buildCpuCeiling(cpuName, gameName, presetName, resolutionName, ramMult) {
  const cpu    = MODEL.cpus[cpuName];
  const game   = MODEL.games[gameName];
  const preset = game.presets[presetName];
  const res    = MODEL.resolutions[resolutionName];

  // La cache 3D beneficia mas en juegos con alto x3dWeight
  const x3dBonus = cpu.cacheClass === "x3d"
    ? game.x3dWeight * cpu.cacheBonus
    : 0;

  // Nucleos extra sobre 6 ayudan en 1% lows (mejor scheduling, menos stalls)
  const coreBonus = Math.max(cpu.cores - 6, 0) * 0.014;

  // A mas resolucion, el cuello de CPU importa menos (GPU absorbe mas trabajo)
  // cpuScale: 1080p=1.0, 1440p=0.95, 4K=0.90
  const resCpuScale = res.cpuScale;

  const avgRaw = preset.cpuBaseline.avg
    * cpu.cpuFactor
    * (1 + x3dBonus)
    * ramMult.avgMult
    * resCpuScale;

  const lowRaw = preset.cpuBaseline.low1
    * cpu.lowFactor
    * (1 + x3dBonus * 1.08)   // X3D mejora mas los lows (frame time variance)
    * (1 + coreBonus)
    * ramMult.lowMult
    * resCpuScale;

  return {
    avg:  clamp(Math.round(avgRaw),  preset.cpuClamp.min, preset.cpuClamp.max),
```

```
    low1: clamp(Math.round(lowRaw),  Math.round(preset.cpuClamp.min * 0.68),
Math.round(preset.cpuClamp.max * 0.86)),
  };
}

// Combina ambos techos en FPS final y deriva todos los indicadores.
// La clave: gpuWeight del juego determina cuanto "pesa" cada techo.
function combineAndDerive(cpuCeiling, gpuCeiling, game, resolutionName, gpuProfile) {
  // gameCpuWeight: que fraccion del rendimiento final depende del CPU
  // CPU-bound=0.75, Mixed=0.50, GPU-bound=0.25
  const cpuWeightMap  = { "CPU-bound": 0.75, Mixed: 0.50, "GPU-bound": 0.25 };
  const cpuW = cpuWeightMap[game.category];
  const gpuW = 1 - cpuW;

  // FPS final: media ponderada de ambos techos
  // Esto evita el "min() duro" que sobreestimaba bottleneck
  const blendedAvg = cpuCeiling.avg * cpuW + gpuCeiling.avg * gpuW;
  const blendedLow = cpuCeiling.low1 * cpuW + gpuCeiling.low1 * gpuW;

  // Penalizacion adicional cuando hay desbalance severo:
  // si un componente supera al otro en >40%, el mas lento tracciona fuerte
  const avgRatio = cpuCeiling.avg / Math.max(gpuCeiling.avg, 1);
  const imbalancePenalty = avgRatio < 0.6
    ? (0.6 - avgRatio) * 0.55   // CPU muy limitado
    : avgRatio > 1.65
    ? (avgRatio - 1.65) * 0.18  // GPU muy limitado (menos impacto en avg)
    : 0;

  const finalAvg = Math.round(blendedAvg * (1 - imbalancePenalty));
  const finalLow = Math.round(blendedLow * (1 - imbalancePenalty * 1.25));

  // GPU usage real: cuando el CPU limita, la GPU espera y su uso cae
  // gpuUsage = (finalAvg / gpuCeiling.avg) * fullUsage
  // Cuando finalAvg == gpuCeiling.avg => usage == fullUsage (~97-99%)
  // Cuando finalAvg << gpuCeiling.avg => usage cae proporcionalmente
  const usageRaw = (finalAvg / Math.max(gpuCeiling.avg, 1)) * gpuCeiling.fullUsage;
  const gpuUsage = clamp(Math.round(usageRaw), 42, 99);

  // Bottleneck: diferencia relativa entre techos, con signo
  // Positivo = CPU es el cuello (cpuCeiling < gpuCeiling)
  // Negativo = GPU es el cuello
  // Cero = balanceado
  const diff = gpuCeiling.avg - cpuCeiling.avg;
  const maxTecho = Math.max(cpuCeiling.avg, gpuCeiling.avg, 1);
  const bottleneckRaw = (diff / maxTecho) * 100;

  // Diagnostico basado en bottleneck real y uso de GPU real
  // CPU bottleneck: CPU techo < GPU techo en > 8% Y uso de GPU baja
```

```
  // GPU bottleneck: GPU techo <= CPU techo Y uso de GPU esta alto
  // Balanced: la diferencia es chica
  let diagnosis;
  if (bottleneckRaw > 8 && gpuUsage < 88) {
    diagnosis = "CPU bottleneck";
  } else if (bottleneckRaw < -8 && gpuUsage >= 90) {
    diagnosis = "GPU bottleneck";
  } else {
    diagnosis = "Balanced";
  }

  // El % que mostramos en el gauge: magnitud del desequilibrio
  const bottleneckDisplay = Math.abs(Math.round(bottleneckRaw));

  return { finalAvg, finalLow, gpuUsage, bottleneckDisplay, diagnosis };
}

// Lows reales dependen de la varianza de frame time del componente limitante.
// CPU-bound => frame scheduling variable => lows mas bajos relativos al avg
// GPU-bound => frame time mas estable => lows mas cercanos al avg
// X3D cache reduce mucho la varianza en CPU-bound (por eso mejora los lows)
function deriveRealistic1pLow(finalAvg, finalLow, cpuCeiling, gpuCeiling, cpuProfile, game) {
  const cpuLimited = cpuCeiling.avg < gpuCeiling.avg * 0.92;

  if (cpuLimited) {
    // Varianza tipica CPU-bound: 1% low ~ 65-75% del avg
    // X3D reduce varianza: sube el ratio hasta ~78%
    const baseRatio = game.category === "CPU-bound" ? 0.68 : 0.73;
    const x3dBoost  = cpuProfile.cacheClass === "x3d" ? 0.07 : 0;
    const coreBoost = Math.min(Math.max(cpuProfile.cores - 6, 0) * 0.008, 0.04);
    const ratio = baseRatio + x3dBoost + coreBoost;
    return Math.round(finalAvg * ratio);
  }

  // GPU-bound: frame time mas estable, 1% low ~ 78-85% del avg
  const gpuRatio = game.category === "GPU-bound" ? 0.82 : 0.78;
  return Math.round(finalAvg * gpuRatio);
}

function getConfidence(cpuName, gpuName, gameName, ramPenalty) {
  const levels = ["low", "medium", "high"];
  const score  = Math.min(
    levels.indexOf(MODEL.cpus[cpuName].confidence),
    levels.indexOf(MODEL.gpus[gpuName].confidence),
    levels.indexOf(MODEL.games[gameName].confidence)
  ) - ramPenalty;

  return score >= 2 ? "high" : score >= 1 ? "medium" : "low";
```

```
}

function calculateScenario(cpuName, gpuName) {
  const gameName      = gameSelect.value;
  const resolutionName = resolutionSelect.value;
  const ramCapacity   = ramCapacitySelect.value;
  const ramSpeed      = ramSpeedSelect.value;
  const ramChannel    = ramChannelSelect.value;
  const cpuProfile    = MODEL.cpus[cpuName];
  const gpuProfile    = MODEL.gpus[gpuName];
  const game          = MODEL.games[gameName];

  // 1. Multiplicadores de RAM
  const ramMult = getRamMultipliers(cpuProfile, gameName, ramCapacity, ramSpeed,
ramChannel);

  // 2. Techo independiente de GPU (sin considerar CPU)
  const gpuCeiling = buildGpuCeiling(gpuName, gameName, state.preset, resolutionName);

  // 3. Techo del CPU (con RAM aplicada)
  const cpuCeiling = buildCpuCeiling(cpuName, gameName, state.preset, resolutionName,
ramMult);

  // 4. Combinar y derivar FPS final, gpuUsage y diagnostico
  const derived = combineAndDerive(cpuCeiling, gpuCeiling, game, resolutionName,
gpuProfile);

  // 5. 1% lows realistas segun componente limitante
  const realistic1pLow = deriveRealistic1pLow(
    derived.finalAvg,
    derived.finalLow,
    cpuCeiling,
    gpuCeiling,
    cpuProfile,
    game
  );

  // 6. Confianza y rango
  const confidence = getConfidence(cpuName, gpuName, gameName,
ramMult.confidencePenalty);
  const spreadMap  = { high: 0.055, medium: 0.085, low: 0.12 };
  const spread     = spreadMap[confidence];
  const rangeMin   = Math.max(14, Math.round(derived.finalAvg * (1 - spread)));
  const rangeMax   = Math.round(derived.finalAvg * (1 + spread * 0.55));

  return {
    diagnosis:       derived.diagnosis,
    confidence,
```

```
    average:          clamp(derived.finalAvg, 14, 520),
    low1:             clamp(realistic1pLow, 10, 420),
    range:            { min: rangeMin, max: rangeMax },
    gpuUsage:         derived.gpuUsage,
    bottleneckPercent: clamp(derived.bottleneckDisplay, 0, 65),
    // Pasamos los techos para buildReferenceText
    _cpuCeiling:      cpuCeiling,
    _gpuCeiling:      gpuCeiling,
    explanation: buildExplanation(
      cpuProfile,
      gpuProfile,
      game,
      resolutionName,
      derived.diagnosis,
      ramCapacity,
      ramSpeed,
      ramChannel,
      cpuCeiling,
      gpuCeiling,
      derived.gpuUsage
    ),
    reference: buildReferenceText(gameName, cpuCeiling, gpuCeiling),
  };
}

function buildReferenceText(gameName, cpuCeiling, gpuCeiling) {
  const dominante = cpuCeiling.avg < gpuCeiling.avg ? "CPU" : "GPU";
  const diff = Math.abs(gpuCeiling.avg - cpuCeiling.avg);
  const pct  = Math.round((diff / Math.max(gpuCeiling.avg, cpuCeiling.avg, 1)) * 100);
  return `${gameName} — Techo CPU: ${cpuCeiling.avg} FPS avg / ${cpuCeiling.low1} low |
Techo GPU: ${gpuCeiling.avg} FPS avg / ${gpuCeiling.low1} low. Desbalance entre techos:
${pct}% (${dominante} limita). El FPS final combina ambos techos segun el peso CPU/GPU
del juego.`;
}

function buildExplanation(cpu, gpu, game, resolution, diagnosis, ramCapacity, ramSpeed,
ramChannel, cpuCeiling, gpuCeiling, gpuUsage) {
  const ramBad = ramCapacity === "8 GB" || ramChannel === "Single channel";
  const memoryLine = ramBad
    ? `La configuracion de RAM (${ramCapacity}, ${ramSpeed},
${ramChannel.toLowerCase()}) reduce los 1% lows y puede generar stutter en escenas
cargadas.`
    : `La RAM (${ramCapacity}, ${ramSpeed}, ${ramChannel.toLowerCase()}) no es un cuello
relevante en este escenario.`;

  const gpuUsageNote = gpuUsage < 75
    ? `El uso de GPU cayo a ${gpuUsage}% porque el CPU no puede alimentarla rapido.`
    : gpuUsage >= 95
```

```
    ? `La GPU corre al ${gpuUsage}% de uso, completamente saturada.`
    : `La GPU opera al ${gpuUsage}% de uso, con algo de margen disponible.`;

  const reasonByDiagnosis = {
    "CPU bottleneck": `El CPU es el cuello: su techo (${cpuCeiling.avg} FPS) queda por
debajo del techo GPU (${gpuCeiling.avg} FPS). ${cpu.notes}`,
    Balanced:         `Ambos componentes estan bien pareados. ${cpu.notes}`,
    "GPU bottleneck": `La GPU es el cuello: su techo (${gpuCeiling.avg} FPS) limita antes
que el CPU (${cpuCeiling.avg} FPS). ${gpu.notes}`,
  };

  return `${reasonByDiagnosis[diagnosis]} ${gpuUsageNote} En ${resolution} con ajuste
${game.category.toLowerCase()}: ${game.notes} ${memoryLine}`;
}

function getResultClass(diagnosis) {
  if (diagnosis === "CPU bottleneck") return "state-cpu";
  if (diagnosis === "GPU bottleneck") return "state-gpu";
  return "state-balanced";
}

function renderMain() {
  const result = calculateScenario(cpuSelect.value, gpuSelect.value);
  const confidenceLabels = {
    high: "Confianza alta",
    medium: "Confianza media",
    low: "Confianza baja",
  };

  // diagnosis
  if (diagnosisText) diagnosisText.textContent = result.diagnosis;
  const diagBadge = document.getElementById("diagnosis-badge");
  if (diagBadge) {
    const labels = { "CPU bottleneck": "⚠️ CPU limita", Balanced: "✅ Balanceado", "GPU
bottleneck": "🎮 GPU limita" };
    const sp = diagBadge.querySelector("span");
    if (sp) sp.textContent = labels[result.diagnosis] || result.diagnosis;
  }

  // confidence
  if (confidencePill) confidencePill.textContent = confidenceLabels[result.confidence];
  const confDot = document.getElementById("conf-dot");
  if (confDot) confDot.className = "conf-dot" + (result.confidence !== "high" ? " " +
result.confidence : "");

  // numbers
  if (avgFpsValue)    avgFpsValue.textContent    = `${result.average}`;
  if (rangeValue)     rangeValue.textContent     = `${result.range.min} — ${result.range.max}`;
```

```
  if (lowValue)       lowValue.textContent       = `${result.low1}`;
  if (gpuUsageValue)  gpuUsageValue.textContent  = `${result.gpuUsage}%`;
  if (explanationText) explanationText.textContent = result.explanation;
  if (referenceText)  referenceText.textContent  = result.reference;

  resultCard.className = `result-card ${getResultClass(result.diagnosis)}`;
  if (avgBar) avgBar.style.width = `${clamp((result.average / 280) * 100, 4, 100)}%`;
  if (lowBar) lowBar.style.width = `${clamp((result.low1   / 230) * 100, 4, 100)}%`;
  if (gpuBar) gpuBar.style.width = `${result.gpuUsage}%`;

  // bottleneck bar (nuevo HTML)
  const btFill = document.getElementById("bottleneck-fill");
  if (btFill) btFill.style.width = `${clamp(result.bottleneckPercent / 70 * 100, 0, 100)}%`;
  const btPct  = document.getElementById("bottleneck-percent");
  if (btPct)  btPct.textContent  = `${result.bottleneckPercent}%`;

  // gauge viejo (por si existe)
  if (bottleneckGauge) {
    const gaugeNorm = Math.min(result.bottleneckPercent / 70 * 100, 100);
    bottleneckGauge.style.setProperty("--gauge-value", `${gaugeNorm}%`);
    const gMap = { "CPU bottleneck": "var(--red)", Balanced: "var(--yellow)", "GPU
bottleneck": "var(--green)" };
    bottleneckGauge.style.setProperty("--gauge-color", gMap[result.diagnosis]);
  }
}

function createCompareCard(cpuName) {
  const result = calculateScenario(cpuName, gpuSelect.value);
  const card = document.createElement("article");
  card.className = `compare-item ${getResultClass(result.diagnosis)}`;

  card.innerHTML = `
    <h3>${cpuName}</h3>
    <p>${result.explanation}</p>
    <div class="compare-meta">
      <strong>${result.average} FPS promedio</strong>
      <span>Rango: ${result.range.min} - ${result.range.max}</span>
      <span>1% low: ${result.low1}</span>
      <span>Uso de GPU: ${result.gpuUsage}%</span>
      <span>Diagnostico: ${result.diagnosis}</span>
    </div>
  `;

  return card;
}

function renderComparison() {
  compareResults.innerHTML = "";
```

```
  compareResults.appendChild(createCompareCard(compareCpuA.value));
  compareResults.appendChild(createCompareCard(compareCpuB.value));
}

function handleUpdate() {
  renderMain();
  renderComparison();
}

function setPreset(nextPreset) {
  state.preset = nextPreset;
  presetButtons.forEach((button) => {
    button.classList.toggle("is-active", button.dataset.preset === nextPreset);
  });
  handleUpdate();
}

function populateControls() {
  const cpuNames = hardwareOptions.cpus;
  const gpuNames = hardwareOptions.gpus;
  const gameNames = Object.keys(MODEL.games);

  refreshHardwareSelect(cpuSelect, cpuNames, "", "Ryzen 7 5800X3D");
  refreshHardwareSelect(compareCpuA, cpuNames, "", "Ryzen 5 5600");
  refreshHardwareSelect(compareCpuB, cpuNames, "", "Ryzen 7 7800X3D");
  refreshHardwareSelect(gpuSelect, gpuNames, "", "RTX 3080");
  fillSelect(gameSelect, gameNames);
  fillSelect(resolutionSelect, Object.keys(MODEL.resolutions));
  fillSelect(ramCapacitySelect, Object.keys(MODEL.ram.capacity));
  fillSelect(ramSpeedSelect, Object.keys(MODEL.ram.speed));
  fillSelect(ramChannelSelect, Object.keys(MODEL.ram.channel));

  gameSelect.value = "Warzone";
  resolutionSelect.value = "1440p";
  ramCapacitySelect.value = "16 GB";
  ramSpeedSelect.value = "3200 MHz";
  ramChannelSelect.value = "Dual channel";
}

function attachHardwareSearches() {
  const searchBindings = [
    {
      input: cpuSearch,
      select: cpuSelect,
      options: hardwareOptions.cpus,
      type: "cpu",
      onAfterUpdate: handleUpdate,
    },
```

```
  {
    input: gpuSearch,
    select: gpuSelect,
    options: hardwareOptions.gpus,
    type: "gpu",
    onAfterUpdate: handleUpdate,
  },
  {
    input: compareCpuASearch,
    select: compareCpuA,
    options: hardwareOptions.cpus,
    type: "cpu",
    onAfterUpdate: renderComparison,
  },
  {
    input: compareCpuBSearch,
    select: compareCpuB,
    options: hardwareOptions.cpus,
    type: "cpu",
    onAfterUpdate: renderComparison,
  },
];

searchBindings.forEach(({ input, select, options, type, onAfterUpdate }) => {
  const suggestionList = createSuggestionList(input);

  const applySelection = (value) => {
    input.value = value;
    refreshHardwareSelect(select, options, value, value);
    suggestionList.hidden = true;
    onAfterUpdate();
  };

  input.addEventListener("input", () => {
    const currentValue = select.value;
    const hasResults = refreshHardwareSelect(select, options, input.value, currentValue);
    const matches = filterOptions(options, input.value);
    renderSuggestions(suggestionList, matches, type, applySelection);

    if (hasResults && select.value) {
      onAfterUpdate();
    }
  });

  input.addEventListener("focus", () => {
    const matches = filterOptions(options, input.value);
    renderSuggestions(suggestionList, matches, type, applySelection);
  });
```

```
    input.addEventListener("keydown", (event) => {
      if (event.key === "Enter") {
        const matches = filterOptions(options, input.value);
        if (matches.length > 0) {
          event.preventDefault();
          applySelection(matches[0]);
        }
      }

      if (event.key === "Escape") {
        suggestionList.hidden = true;
      }
    });

    input.addEventListener("blur", () => {
      window.setTimeout(() => {
        suggestionList.hidden = true;
      }, 120);
    });
  });
}

function attachEvents() {
  [
    cpuSelect,
    gpuSelect,
    gameSelect,
    resolutionSelect,
    ramCapacitySelect,
    ramSpeedSelect,
    ramChannelSelect,
  ].forEach((element) => {
    element.addEventListener("change", handleUpdate);
  });

  [compareCpuA, compareCpuB].forEach((element) => {
    element.addEventListener("change", renderComparison);
  });

  presetButtons.forEach((button) => {
    button.addEventListener("click", () => setPreset(button.dataset.preset));
  });
}

populateControls();
attachHardwareSearches();
attachEvents();
```

```
handleUpdate();

// ─── CHATBOT FLOTANTE ──────────────────────────────────────────────────────────────────────────
//
// IMPORTANTE: La API de Anthropic requiere una API key enviada desde el servidor.
// Desde un HTML local no funciona por CORS + seguridad de la key.
// Para producción: usar un proxy backend (Cloudflare Worker, Vercel Edge, etc.)
// que agregue el header "x-api-key" y reenvíe la request a api.anthropic.com.
//
──────────────────────────────────────────────────────────────────────────
```

```
  const panel = document.getElementById("chat-panel");
  panel.classList.remove("open");
  panel.setAttribute("aria-hidden", "true");
}

// ── Widget init ──────────────
```

```
  const txt     = input.value.trim();
  if (!txt) return;

  input.value = ""; input.disabled = true; sendBtn.disabled = true;

  // primer mensaje: inyectar contexto de la herramienta
  let content = txt;
  if (!chatHistory.length) {
    content = `[Contexto herramienta: CPU=${cpuSelect.value}, GPU=${gpuSelect.value},
Juego=${gameSelect.value}, Resolución=${resolutionSelect.value},
RAM=${ramCapacitySelect.value} ${ramSpeedSelect.value}
${ramChannelSelect.value}]\n\n${txt}`;
  }

  addMsg(msgs, txt, "user-msg");

  const typing = document.createElement("div");
  typing.className = "msg bot-msg typing-msg";
  typing.innerHTML = "<span></span><span></span><span></span>";
  msgs.appendChild(typing);
  msgs.scrollTop = msgs.scrollHeight;

  chatHistory.push({ role: "user", content });

  try {
    const res  = await fetch("https://api.anthropic.com/v1/messages", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        model: "claude-sonnet-4-20250514",
        max_tokens: 1024,
        system: CHAT_SYSTEM,
        messages: chatHistory,
      }),
    });

    if (!res.ok) {
      const err = await res.json().catch(() => ({}));
      throw new Error(err.error?.message || `HTTP ${res.status}`);
    }

    const data  = await res.json();
    const reply = data.content.filter(b => b.type === "text").map(b => b.text).join("");
    chatHistory.push({ role: "assistant", content: reply });
    typing.remove();
    addMsg(msgs, reply, "bot-msg");

  } catch (err) {
```

```
    typing.remove();
    const isLocal = location.protocol === "file:";
    const msg = isLocal
      ? "El chatbot requiere que el sitio esté hosteado en un servidor web con proxy backend
para la API key. No funciona con file://."
      : `Error de conexión: ${err.message}`;
    addMsg(msgs, msg, "bot-msg error-msg");
  }

  msgs.scrollTop = msgs.scrollHeight;
  input.disabled = false; sendBtn.disabled = false; input.focus();
}

function addMsg(container, text, classes) {
  const el = document.createElement("div");
  el.className = `msg ${classes}`;
  el.textContent = text;
  container.appendChild(el);
}

function initChatbot() {
  document.getElementById("chatbot-send").addEventListener("click", sendChatMessage);
  document.getElementById("chatbot-input").addEventListener("keydown", e => {
    if (e.key === "Enter" && !e.shiftKey) { e.preventDefault(); sendChatMessage(); }
  });
  document.querySelectorAll(".chat-chip").forEach(chip => {
    chip.addEventListener("click", () => {
      document.getElementById("chatbot-input").value = chip.textContent.trim();
      if (!chatOpen) openChat();
      sendChatMessage();
    });
  });
}

initChatWidget();
```